

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №4

по дисциплине

«Микропроцессорные и встраиваемые системы»

Выполнили: **Салженикин Д.А.**

Галиев Э.Р.

Группа: **ИУ1-52Б**

Проверил: **Сычёв В.В.**

Работа выполнена: **23.11.2023**

Отчет сдан: **14.12.2023**

Оценка:

Москва 2023

Содержание

Цель работы: 3
Теоретическая часть 3
Принцип работы ПИД-регулятора: 4
Практическая часть 5
Код 5
Реализация ПИ-регулятора в коде: 7

Цель работы:

Регулирование скорости вращения мотора с помощью ПИ-регулятора.

Теоретическая часть

Регуляторы - это устройства, которые используются для управления или регулирования процессов в различных системах. Они могут быть использованы в электронике, автомобильной промышленности, отоплении и кондиционировании воздуха, промышленных процессах и других областях.

Пропорционально-интегрально-дифференциальный (ПИД) регулятор является одним из самых распространенных типов регуляторов, используемых в промышленности и автоматизированных системах. Он объединяет в себе три основных компонента: пропорциональный, интегральный и дифференциальный, каждый из которых вносит свой вклад в управление процессом. Они могут быть настроены и скомбинированы различными способами в зависимости от конкретной системы и требуемых характеристик управления.

Пропорциональный компонент обеспечивает реакцию на текущую ошибку между уставкой и фактическим значением, пропорционально этой ошибке. Это позволяет быстро реагировать на изменения в системе и уменьшить перерегулирование.

Интегральный компонент накапливает ошибку во времени и корректирует управление, чтобы устранить статическую ошибку. Это помогает достичь точного управления и устранить постоянную ошибку в системе.

Дифференциальный компонент предотвращает перерегулирование и улучшает стабильность системы, предсказывая изменения входного сигнала и корректируя управление заранее.

Настройка ПИД-регулятора включает определение коэффициентов каждого компонента.

Соответственно ПИ-регулятор – регулятор, в котором активны 2 компонента: пропорциональный и интегральный. Пропорционально-интегральный регулятор реагирует на текущую ошибку пропорционально ей же, а также накапливает ошибку во времени и корректирует управление, чтобы

устранить статическую ошибку. Это позволяет достичь точного управления и устранить постоянную ошибку в системе.

Принцип работы ПИД-регулятора:

ПИД-регулятор формирует управляющий сигнал, являющийся суммой трёх слагаемых: первое – пропорциональная разность входного сигнала и сигнала обратной связи (сигнал рассогласования/ошибка управления), второе — интеграл сигнала рассогласования, третье — производная сигнала рассогласования. Рисунок 1 – иллюстрация работы ПИД-регулятора.

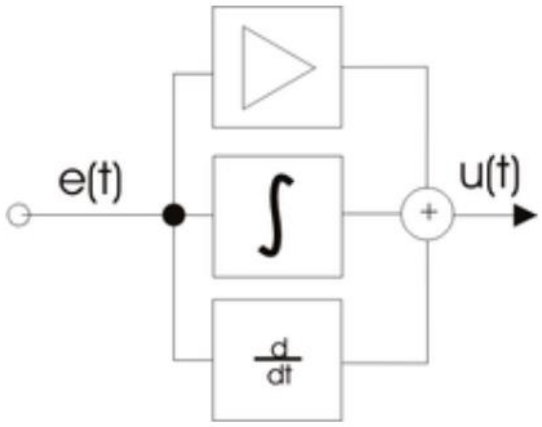


Рис. 1. Принцип работы ПИД-регулятора

Формулы для формирования управляющего сигнала:

Определение пропорциональной компоненты:

$$P(t) = K_p \times e(t)$$

Определение интегральной компоненты:

$$I(t) = K_i \times \int \quad e(t)dt$$

Определение дифференциальной компоненты:

$$D(t) = \frac{K_d de(t)}{dt}$$

Таким образом, наладка ПИД-регулятора сводится к определению коэффициентов Kp, Ki, Kd.

Практическая часть

Код

```
/* USER CODE BEGIN PV */
unsigned long countOfTIMoverflow;
unsigned long  currentStateTIM;
unsigned long encoderPeriod;
unsigned long oldStateTIM;
unsigned long angle;
double speed[100], needSpeed;
double intErr,reg;
double err;
/* USER CODE END PV */
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_TIM1_Init(void);
static void MX_USART2_UART_Init(void);
int main(void) {
  /* USER CODE BEGIN 1 */
  /* USER CODE END 1 */
  /* MCU Configuration--------------------------------------------------------*/
  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();
  /* USER CODE BEGIN Init */
  /* USER CODE END Init */
  /* Configure the system clock */
  SystemClock_Config();
  /* USER CODE BEGIN SysInit */
  /* USER CODE END SysInit */
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_ADC1_Init();
  MX_TIM1_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */
  countOfTIMoverflow = 0;
  currentStateTIM = 0;
  oldStateTIM = 0;
  reg=0;
  uint32_t adcValue;
  uint32_t str[31];
  HAL_GPIO_WritePin(GPIOE,GPIO_PIN_10,GPIO_PIN_SET);
  HAL_GPIO_WritePin(GPIOE,GPIO_PIN_12,GPIO_PIN_RESET);
  uint32_t temp;
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
  /* USER CODE END 2 */
```

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1) {
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1,10);
    adcValue = HAL_ADC_GetValue(&hadc1);
    //TIM1->CCR2 = adcValue;
    needSpeed = (adcValue - 2047) / 5;
    HAL_Delay(40);
    err = needSpeed - speed[98];
    intErr = intErr + err * encoderPeriod;
    reg = 3*intErr + err * 5;
    TIM1->CCR2 = (uint16_t)reg;
    /* if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==GPIO_PIN_RESET)
    {
              temp = TIM1->CCR2;
              TIM1->CCR2 = 0;
              HAL_GPIO_TogglePin(GPIOE,GPIO_PIN_10);
              HAL_GPIO_TogglePin(GPIOE,GPIO_PIN_12);
              TIM1->CCR2 = temp;
    }*/
   /* USER CODE END WHILE */
   /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
  * @brief System Clock Configuration
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
  double summ=0;
  oldStateTIM = currentStateTIM;
  currentStateTIM =currentStateTIM + TIM1->CNT;
  encoderPeriod = currentStateTIM - oldStateTIM;
  for (int i=0; i<98; i++){
            speed[i]=speed[i+1];
  }
  for (int i=0; i<98; i++){
            summ = summ + speed[i];
  }
  if(HAL_GPIO_ReadPin(GPIOE,GPIO_PIN_14)==GPIO_PIN_SET) {
            speed[98] = ((-10000/ encoderPeriod)+summ)/98;
            angle++;
  }
  else {    speed[98] = ((-10000 / encoderPeriod)+summ)/98;
            angle--;
  }
}
/* USER CODE END 4 */
```

Реализация ПИ-регулятора в коде:

Пропорциональный модуль:

P = setpoint - input

Интегрирующий модуль:

I = I + (setpoint - input) * dt